Problem Solving and Search

Overview

- Problem solving agents
- Problem types
- Problem formulation
- Example problems
- Basic search algorithms

Problem-Solving Agents

- Goal Formulation
- Problem Formulation
- Solution/objective

Building Goal-Based Agents

- We have a **goal** to reach (Goal formulation)
  - Driving from point A to point B
  - Put 8 queens on a chess board such that no one attacks another
  - Prove that John is an ancestor of Mary
- We have information about where we are now at the **beginning/initial state (problem formulation)**
- We have a set of **actions** we can take to move around (change from where we are) [problem formulation)
- **Objective/Solution**: find a sequence of **legal actions** which will bring us from the start point to a goal

What is the goal to be achieved?

- Could describe a **situation** we want to achieve, a set of **properties** that we want to hold, etc.
- Requires defining a **"goal test"** so that we know what it means to have achieved/satisfied our goal.
- This is a hard part that is rarely tackled in AI, usually assuming that the system designer or user will specify the goal to be achieved.

What are the actions?

- Quantify all of the primitive actions or events that are sufficient to describe all necessary changes in solving a task/goal.
- **Deterministic**: No uncertainty in an action's effect. Given an action (**operator** or **move**) and a description of the current **state of the world**, the action completely specifies
  - **Precondition:** if that action CAN be applied to the current world (i.e., is it applicable and legal), and
  - **Effect:** what the exact state of the world will be after the action is performed in the current world (i.e., no need for "history" information to be able to compute what the new world looks like).

Representing Actions

- Note also that actions can all be considered as **discrete events** that can be thought of as occurring at an **instant of time**.
  - That is, the world is in one situation, then an action occurs and the world is now in a new situation. For example, if "Mary is in class" and then performs the action "go home," then in the next situation she is "at home." There is no representation of a point in time where she is neither in class nor at home (i.e., in the state of "going home").
- The number of operators needed depends on the **representation** used in describing a state.
  - In the 8-puzzle, we could specify 4 possible moves for each of the 8 tiles, resulting in a total of **4*8=32 operators**.
  - On the other hand, we could specify four moves for the "blank" square and we would only need **4 operators**.

Representing states

- At any moment, the relevant world is represented as a **state**
  - Initial (start) state: S
  - An action (or an operation) changes the current state to another state (if it is applied): state transition
  - An action can be taken (applicable) only if its precondition is met by the current state
  - For a given state, there might be more than one applicable actions
  - **Goal state**: a state satisfies the goal description or passes the goal test
  - **Dead-end state**: a non-goal state to which no action is applicable

Representing states

- **State space**:
  - Includes the initial state S and all other states that are reachable from S by a **sequence of actions**
  - A state space can be organized as a **graph**:

    nodes: states in the space

    arcs: actions/operations
- The **size of a problem** is usually described in terms of the **number of states** (or the size of the state space) that are possible.
  - Tic-Tac-Toe has about 3^9 states.
  - Checkers has about 10^40 states.
  - Rubik's Cube has about 10^19 states.
  - Chess has about 10^120 states in a typical game.
  - GO has more states than Chess

Representing states

- **State space**:
  - Includes the initial state S and all other states that are reachable from S by a **sequence of actions**
  - A state space can be organized as a **graph**:

    nodes: states in the space

    arcs: actions/operations
- The **size of a problem** is usually described in terms of the **number of states** (or the size of the state space) that are possible.
  - Tic-Tac-Toe has about 3^9 states.
  - Checkers has about 10^40 states.
  - Rubik's Cube has about 10^19 states.
  - Chess has about 10^120 states in a typical game.
  - GO has more states than Chess

Offline Problem-Solving Agent

function SIMPLE-PROBLEM-SOLVING-AGENT(p) **returns** action
  inputs : p, a percept
  static: s, an action sequence, initially empty
  *stat*e, some description of the current world state
  g, a goal, initially null
  *proble*m, a problem formulation
 state ← UPDATE-STATE(*state,* p)
 if *s* is empty **then**
  g ← FORMULATE-GOAL(*state*)
  *proble*m ← FORMULATE-PROBLEM(*state,* g)
  s ← SEARCH(*proble*m)
action ← RECOMMENDATION(*s, state*)
s ← REMAINDER(*s, state*)
return *action*

Example Problem: Romania

On holiday in Romania; currently in Arad

Flight leaves tomorrow from Bucharest back home.

Formulate Goa*l:* be in Bucharest

Formulate Problem:

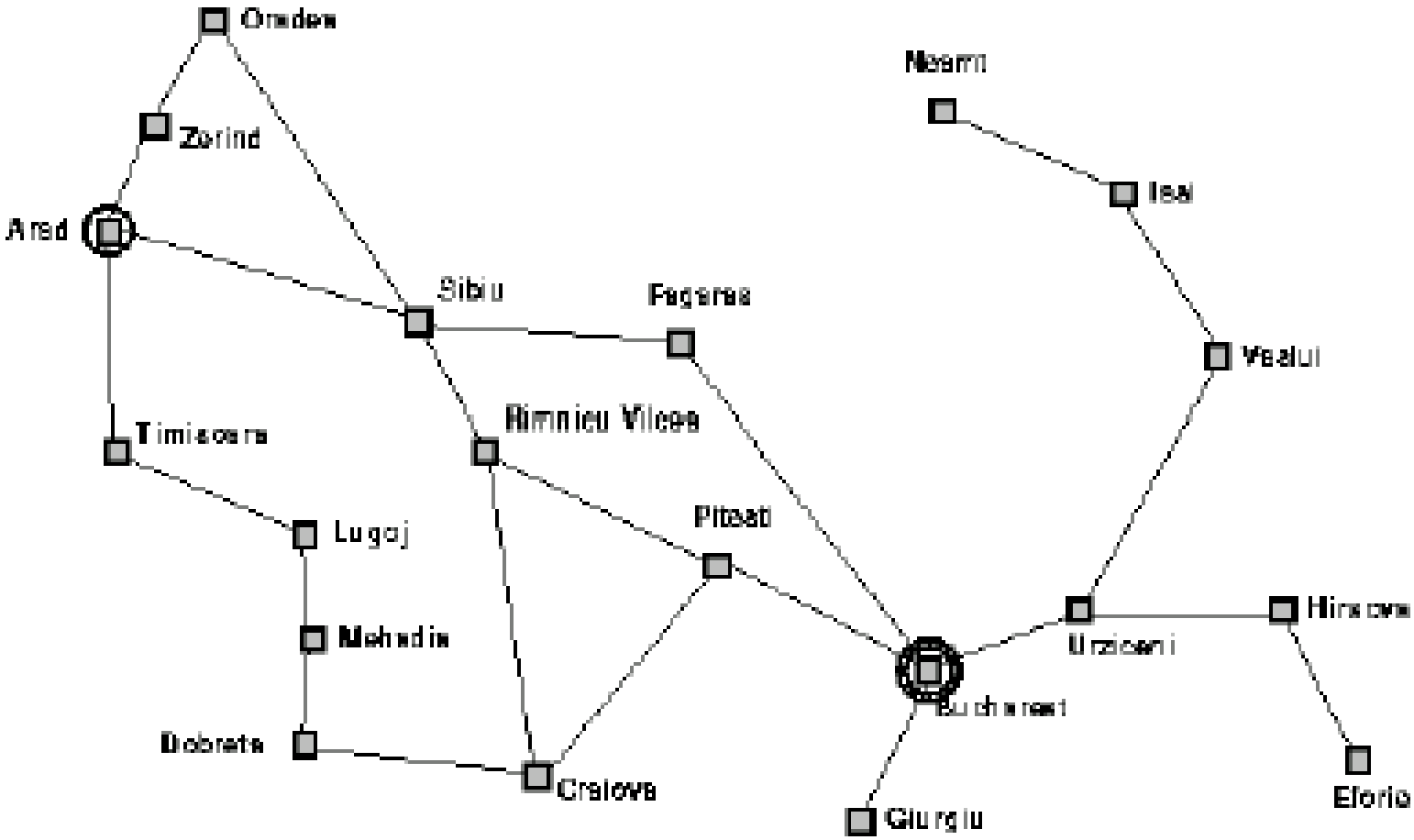
states: various cities

operators: drive between cities

Find Solution:

sequence of cities, e.g. Arad, Sibiu, Fagaras, Bucharest

Example Romania



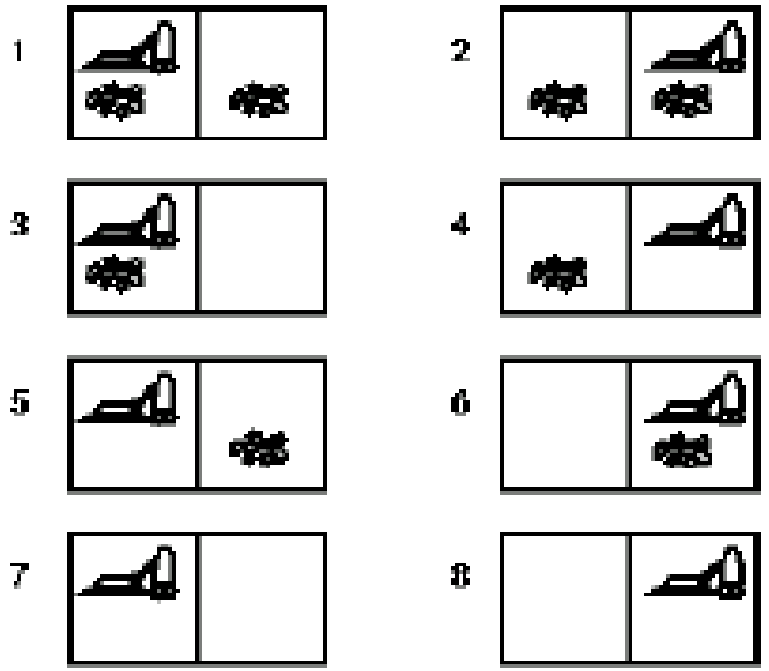
Problem Types

- Deterministic, accessible → *single-state problem*
- Deterministic, inaccessible → *multiple-state problem*
- Nondeterministic, inaccessible → *contingency problem*
  - must use sensors during execution
  - solution is a *tree* or *policy*
  - often *interleave* search, execution
- Unknown state space → *exploration problem*

Problem Types

- Let the world contain just two locations
- Each location may or may not contain dirt and the agent may be in one location or the other
- How many possible world states
- Possible actions : Left, Right, and Suck
- The goal is to clean up all the dirt
- Find a goal state?

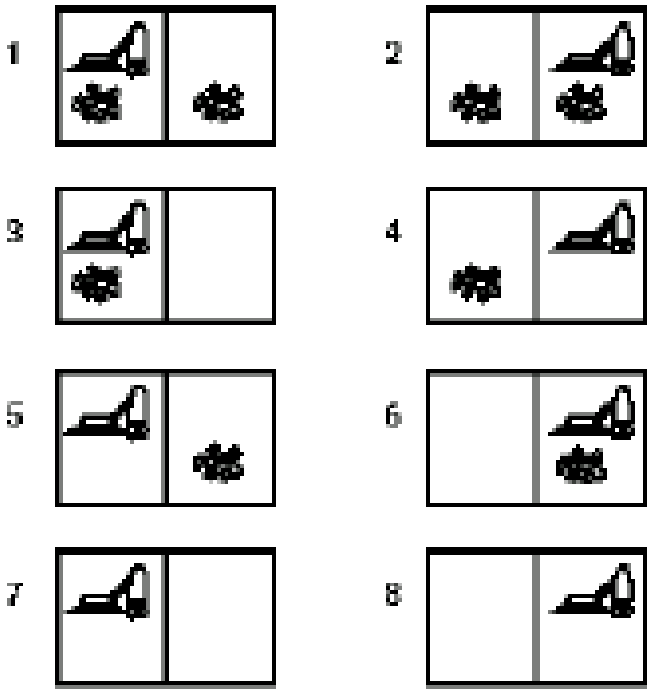
Example: Vacuum world



Single-state, start #5. Solution: ??

Example: vacuum world

- Single-state, start #5. Solution: ??
- Multiple-State, start in {1,2,3,4,5,6,7,8}
  e.g. *Right* goes to {2,4,6,8}
  Solution: ??
- Contingency, start in #5
  Murphy's Law: *Suck* can dirty a clean carpet.
- Local sensing: dirt, location only.
  Solution: ??



Single-state problem formulation

- A *problem* is defined by four items:
- *Initial state* e.g. ‘at Arad’
- *operators* (or successor function S(x)) [Given a particular state x, S(x) returns the set of states reachable from x by any single action]

- *goal tes*t, can be
  - *explici*t, e.g. x=‘at Bucharest’
  - *implici*t, e.g. NoDirt(x)
- *path cost* (additive)

e.g. sum of distances, number of operators executed, etc.

- A *solution* is a sequence of operators leading from the initial state to the goal state

Single-State Problem Continue)

- Datatype PROBLEM
  - Componnets: INITIAL-STATE, OPERATORS, GOAL-TEST, PATH-COST-FUNCTION
- Instance of this datatype will be the input to the serach algorithms
- The output of a serach algorithm is a solution, that is, a path from the initial state to a state that satisfies the goal cost.
- Multiple-state problem? Read (Page 60)

Performance

- The effectiveness of a search can be measured in at least three ways
  - First, does it find a solution at all?
  - Second, is it a good solution (one with a low path cost)?
  - Third, what is the search cost associated with the time and memory required to find a solution?
  - The total cost of the search is the sum of the path cost and the search cost.

Knowledge representation issues

- What's in a state ?
  - Is sunspot activity relevant to predicting the stock market? What to represent is a very hard problem that is usually left to the system designer to specify.
- What **level of abstraction** or detail to describe the world.
  - Too fine-grained and we'll "miss the forest for the trees." Too coarse-grained and we'll miss critical details for solving the problem.
- The number of states depends on the representation and level of abstraction chosen.

Selecting a state space

- Real world is absurdly complex.

  → state space must be *abstracted* for problem solving

  (Abstract) state = set of real states

  (Abstract) operator = complex combination of real actions

  e.g. Arad → Zerind represents a complex set of possible ways to get from somewhere in Arad to somewhere in Zerind.

  (Abstract) solution= set of real paths that are solutions in the real world

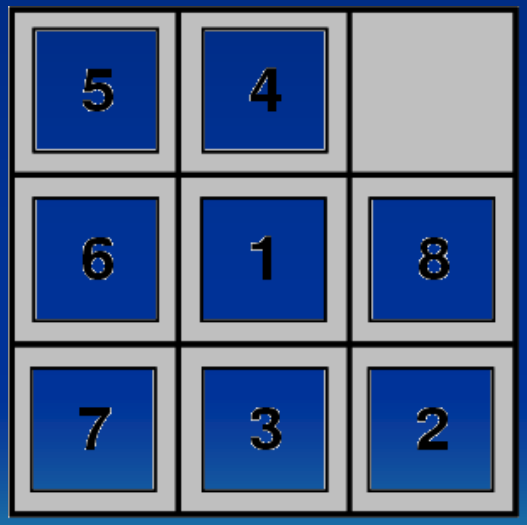
  Each abstract action should be 'easier' than the original problem!

Some example problems

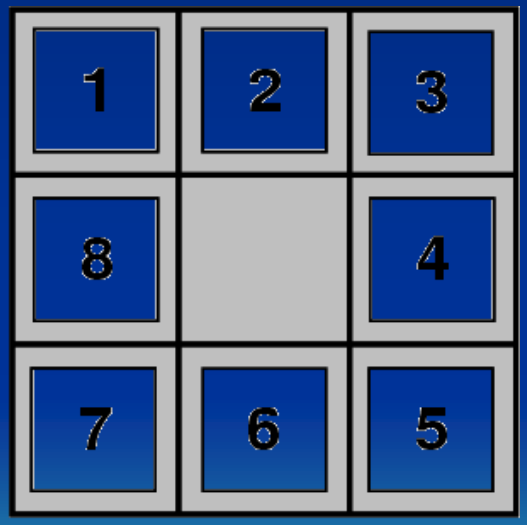
- Toy problems and micro-worlds
  - 8-Puzzle
  - Missionaries and Cannibals
  - Cryptarithmetic
  - Remove 5 Sticks
  - Water Jug Problem
  - Counterfeit Coin Problem
- Real-world-problems

8-Puzzle

Given an initial configuration of 8 numbered tiles on a 3 x 3 board, move the tiles in such a way so as to produce a desired goal configuration of the tiles.



Start State

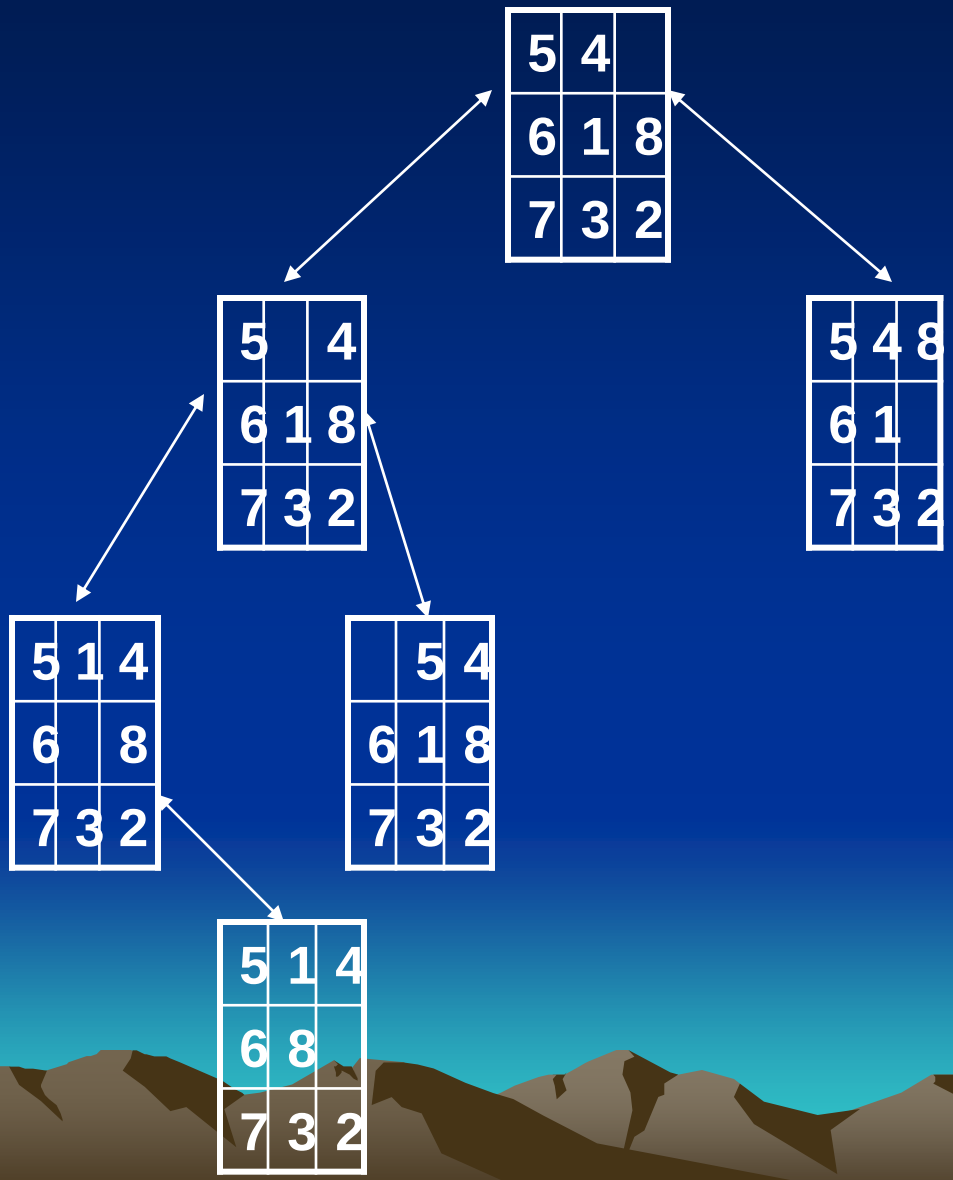


Goal State

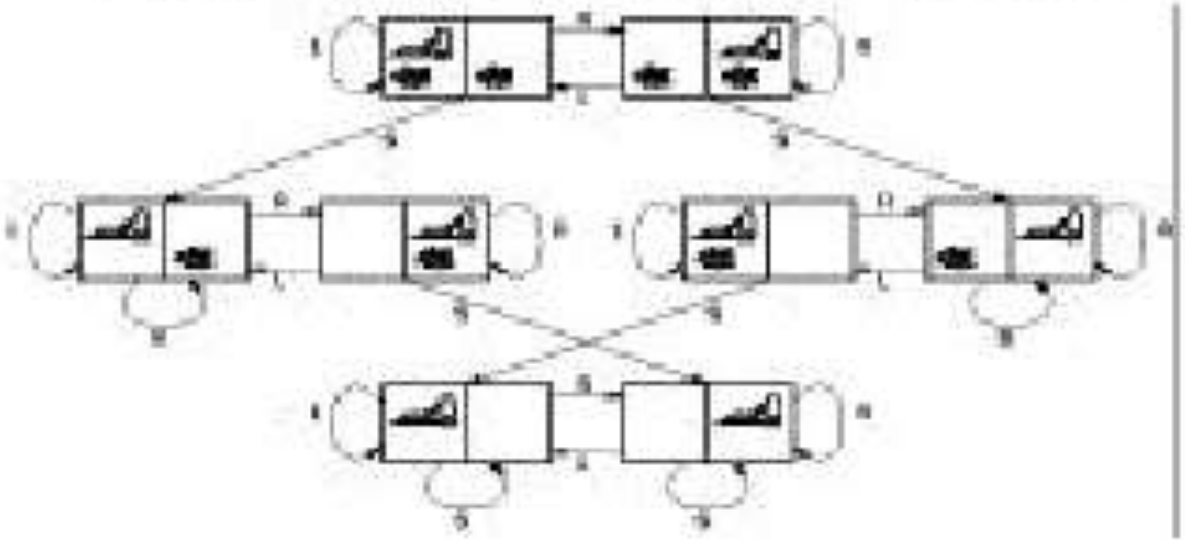
8 Puzzle

- **State:** 3 x 3 array configuration of the tiles on the board (a state description specifies the **location** of each of the eight tiles in one of the nine squares)
- **Operators:** Move Blank square Left, Right, Up or Down.
- **Initial State:** A particular configuration of the board.
- **Goal:** A particular configuration of the board.
- Path Cost: each step costs 1, so the path cost is just the length of the path

A portion of the state space of a 8-Puzzle problem

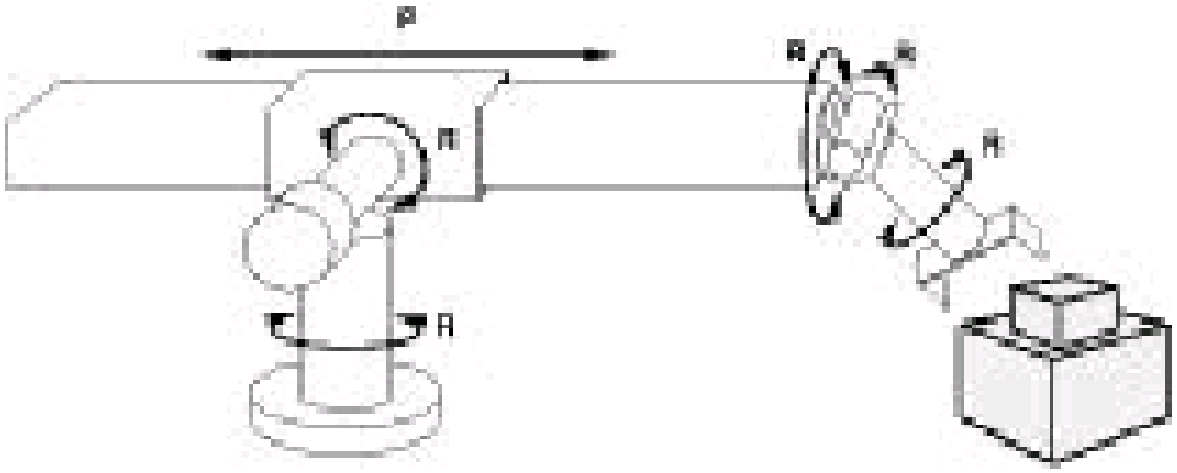


Example: Vacuum world



States:	Dirt and robot location
Operators:	*Left, Right, Suck*
Goal test:	No dirt
Path cost:	1 per operator

Example: Assembly Robot



States: Continuous angles of joints (location of objects)

Operators: *Continuous movement of joints*

Goal test: Complete assembly (without robot included!).

Path cost: Consumed time.

Assignment

- Explain why problem formulation must follow goal formulation.
- Consider the accessible, two-location vacuum world under Murphy's Law. Show that for each initial state, there is a sequence of actions that is guaranteed to reach a goal state.
- Give the initial state, goal test, operators, and path cost function for each of the following. There are several possible formulations for each problem, with varying levels of detail. The main thing is that your formulations should be precise and "hang together" so that they could be implemented.
    - You want to find the telephone number of Mr. Jimwill Zollicoffer, who lives in Alameda, given a stack of directories alphabetically ordered by city
    - As part (a), but you have forgotten Jimwill's last name
    - You are lost in the Amazon jungle, and have to reach the sea. There is a stream nearby.
    - You have to color a complex planar map using only four colors, with no two adjacent regions to have the same color.
    - A monkey is in a room with a crate, with bananas suspended just out of reach on the ceiling. He would like to get the bananas.
    - You are lost in a small country town, and must find a drug store before your hay fever becomes intolerable. There are no maps, and the natives are all located indoors.
-

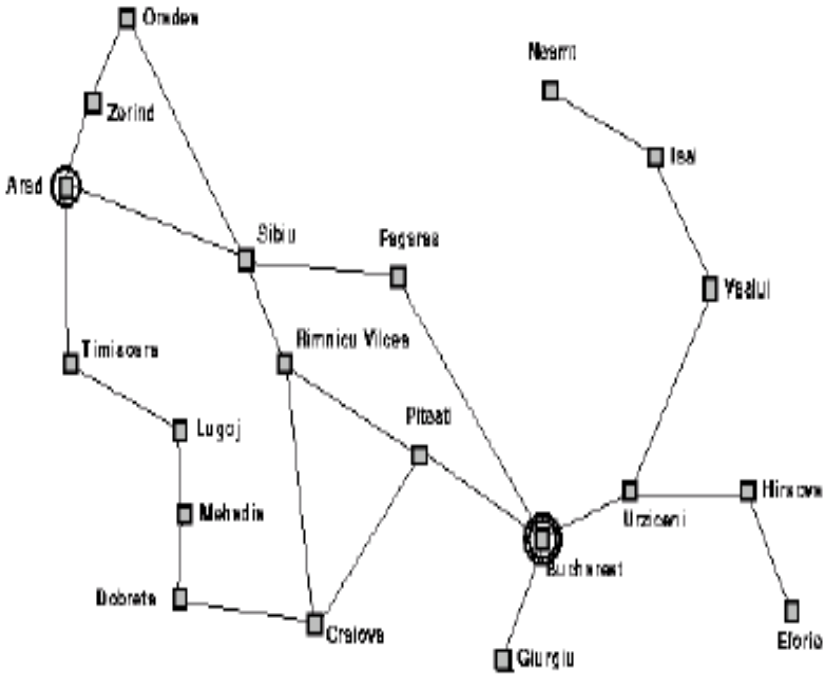
Searching for Solutions

- Problem definition and the recognition of its solution presented
- Finding a solution – is done by a search through state space
- The idea is to maintain and extend a set of partial solution sequences
- The generation of these sequences and how to keep track of them using suitable data structure will be discussed

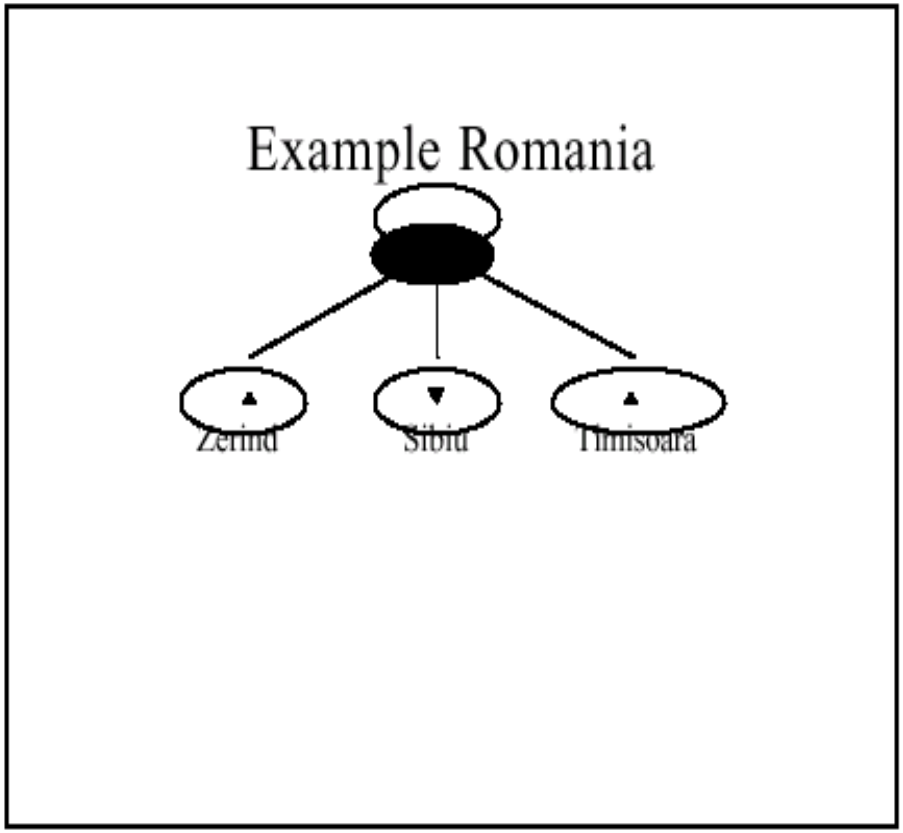
Generating Action Sequences

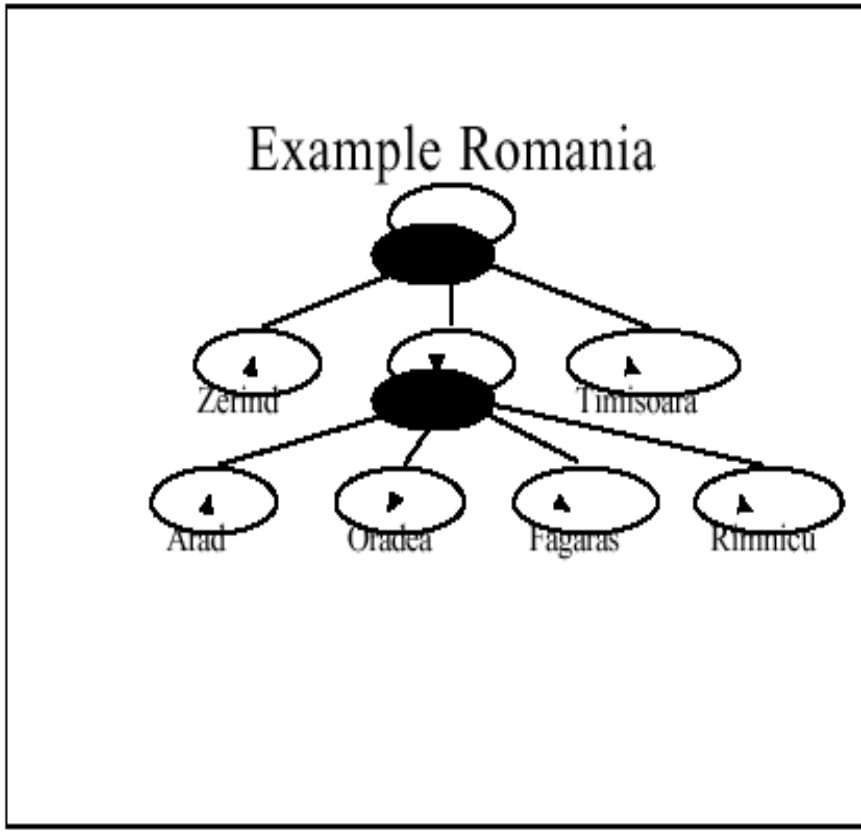
- Expanding the state
- The choice of which state to expand is determined by the search strategy
- The search process consists of building up a search tree that is superimposed over the state space
- **The root of the search tree is a search node corresponding to the initial state**

Example Romania

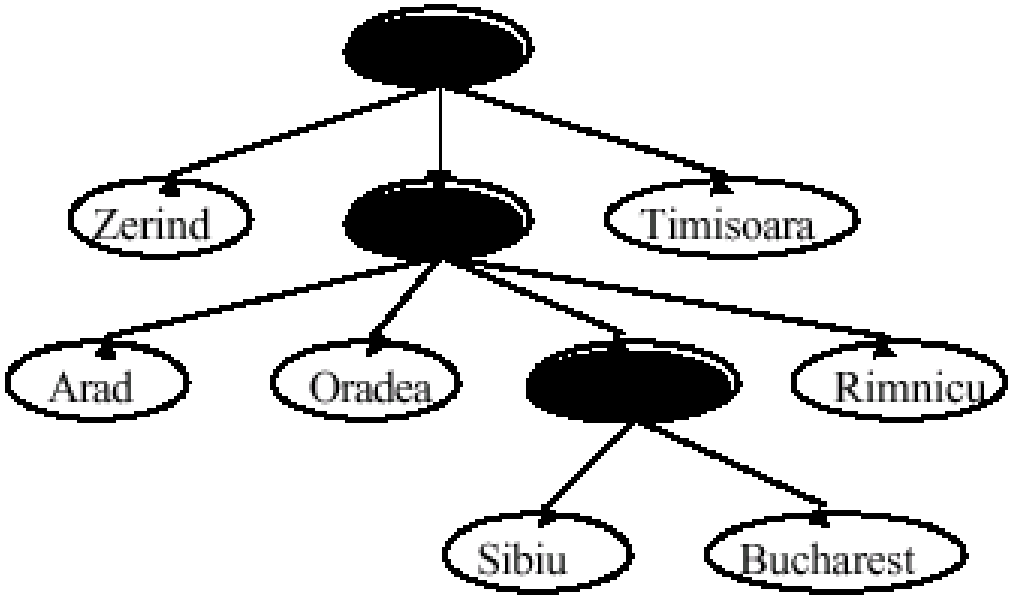


Example Romania


Example Romania
Zerind
Sibiu
Timisoara


Example Romania
Zerind
Timisoara
Arad
Oradea
Fagaras
Rimnicu

Example Romania



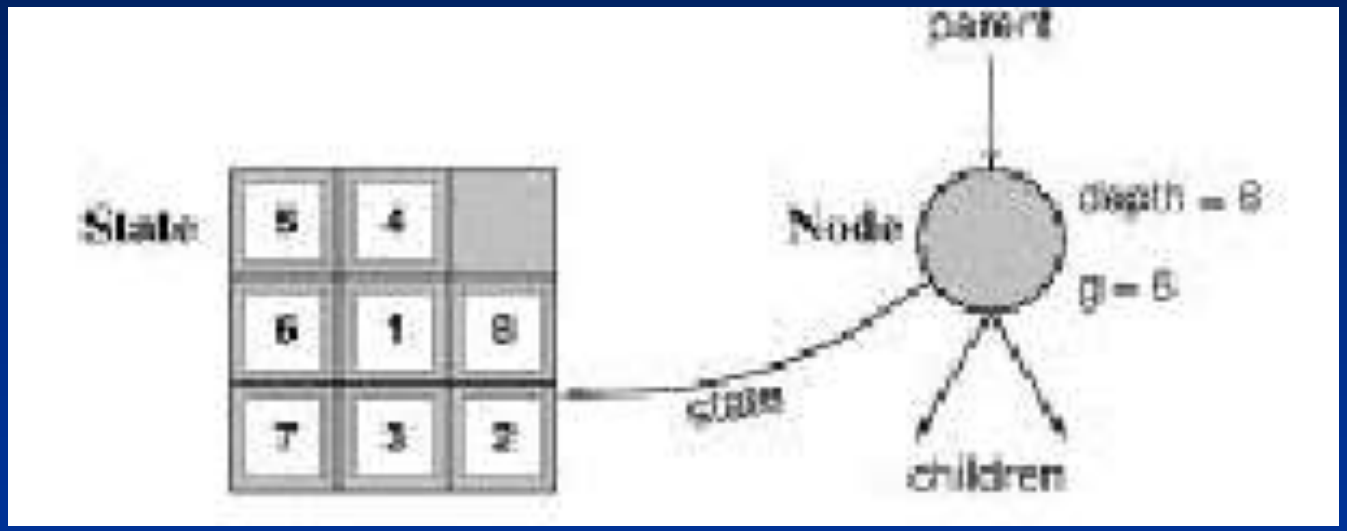
Search Algorithms

- Basic idea: offline exploration of state space
  by simulation.

General-Search(*problem, strategy*) **return** solution/failure
  Initialise search tree with provided *problem*
  loop do
    if no candidates for expansion **return** failure
    choose a leaf node *n* for expansion according to *strategy*
    if *n* contains goal state **then** return corresponding solution
    else expand *n* and add resulting nodes to search tree
  end.

Implementation cont'd: states vs. nodes

- A *state* is a representation of a (physical) configuration
- A *node* is a data structure constituting part of a search tree that includes *parent, children, depth, path cost g(n)*
- *States* do not have *parents, children ,depth, or path cost.*



The EXPAND function creates new *nodes*, filling in the various fields and using the OPERATORS (or SUCCESSOR-Fn) of the problem to create corresponding *states*.

Implementation of Search Algorithms

Function General-Search (*problem, Queueing-fn*) **return** solution/failure

nodes ← MAKE-QUEUE (MAKE-NODE (Initial-State [*problem]*))
loop do
    if *nodes* is empty then **return** failure
    node ← REMOVE-FRONT (*nodes*)

    if (GOAL-TEST (*problem*) applied to STATE (*node*) succeeds
    then return *node*

    nodes ← QUEUEING-FN (*nodes*, EXPAND (*node*,
                OPERATORS [problem])
end.

Basic Search Algorithm

Let fringe be a list containing the initial state

Loop

if fringe is empty return failure
Node $\rightarrow$ remove first (fringe)
if Node is a goal
then return the path from initial state to Node
else generate all successors of node
merge the newly generated nodes into fringe

End

Formalizing Search in a State Space

- A state space is a directed **graph**, (V, E) where V is a set of **nodes** and E is a set of **arcs**, where each arc is directed from a node to another node
- **node:** a **state**
  - state description
  - plus optionally other information related to the parent of the node, operation used to generate the node from that parent, and other bookkeeping data
- **arc:** an instance of an (applicable) action/operation.
  - the source and destination nodes are called as **parent (immediate predecessor)** and **child** (**immediate successor**) nodes with respect to each other
  - ancestors((predecessors) and descendents (successors)
  - each arc has a fixed, non-negative **cost** associated with it, corresponding to the cost of the action

- **node generation:** making explicit a node by applying an action to another node which has been made explicit
- **node expansion:** generate **all** children of an explicit node by applying **all** applicable operations to that node
- One or more nodes are designated as **start nodes**
- A **goal test** predicate is applied to a node to determine if its associated state is a goal state
- A **solution** is a sequence of operations that is associated with a path in a state space from a start node to a goal node
- The **cost of a solution** is the sum of the arc costs on the solution path

- **State-space search** is the process of searching through a state space for a solution by **making explicit** a sufficient portion of an **implicit** state-space graph to include a goal node.
  - Initially V={S}, where S is the start node; when S is expanded, its successors are generated and those nodes are added to V and the associated arcs are added to E. This process continues until a goal node is generated (included in V) and identified (by goal test)
- During search, a node can be in one of the three categories:
  - Not generated yet (has not been made explicit yet)
  - **OPEN**: generated but not expanded
  - **CLOSED**: expanded
- Search strategies differ mainly on how to select an OPEN node for expansion at each step of search

A General State-Space Search Algorithm

- Node n
  - state description
  - parent (if needed)
  - Operator used to generate n (optional)
  - Depth of n (optional)
  - Path cost from S to n (if available)
- OPEN list
  - initialization: {S}
  - node insertion/removal depends on specific search strategy
- CLOSED list
  - initialization: {}

A General State-Space Search Algorithm

open := {S}; closed :={};
repeat
    n := *select* (open); /* select one node from open for expansion */
    if n is a goal
        then exit with success; /* delayed goal testing */
    expand (n) /* generate all children of n put these newly generated nodes in open (check duplicates) put n in closed (check duplicates) */
until open = {};
exit with failure

Some Issues

- Search process constructs a search tree, where
  - **root** is the initial state S, and
  - **leaf nodes** are nodes
    - not yet been expanded (i.e., they are in OPEN list) or
    - having no successors (i.e., they're "deadends")
- Search tree may be infinite because of loops, even if state space is small
- Search strategies mainly differ on *select* (open)
- Each node represents a partial solution path (and cost of the partial solution path) from the start node to the given node.
  - in general, from this node there are many possible paths (and therefore solutions) that have this partial path as a prefix.

Search Strategies

- A search strategy determines the order in which nodes are expanded.
- Dimensions along which strategies are assessed:
  - Completeness: is a solution always found, if one exists?
  - Time complexity: number of nodes expanded/generated
  - Space complexity: number of nodes in memory
  - Optimality: Will a best solution be found?

Evaluating Search Strategies

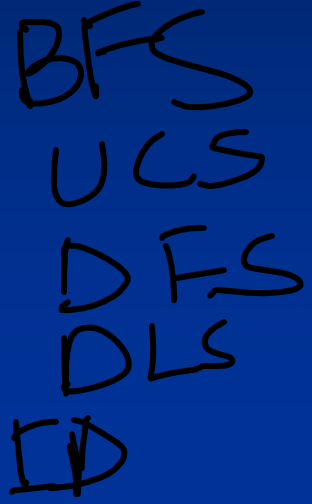
1. **Completeness**
   - Guarantees finding a solution whenever one exists
2. **Time Complexity**
   - How long (worst or average case) does it take to find a solution? Usually measured in terms of the **number of nodes expanded**
3. **Space Complexity**
   - How much space is used by the algorithm? Usually measured in terms of the **maximum size that the "OPEN" list** becomes during the search
4. **Optimality/Admissibility**
   - If a solution is found, is it guaranteed to be an optimal one? For example, is it the one with minimum cost?

Search Strategies

- Time and space complexity are measured in terms of
  - *b*: the branching factor
  - *d*: the depth of the least cost solution
  - *m*: the maximal depth to which the strategy will be searching - this may also be infinite.

Uninformed Search Strategies

- They use only the information available in the problem definition.
  - Breadth first search
  - Uniform cost search
  - Depth first search
  - Depth-limited search
  - Iterative deepening



For each of these student will learn

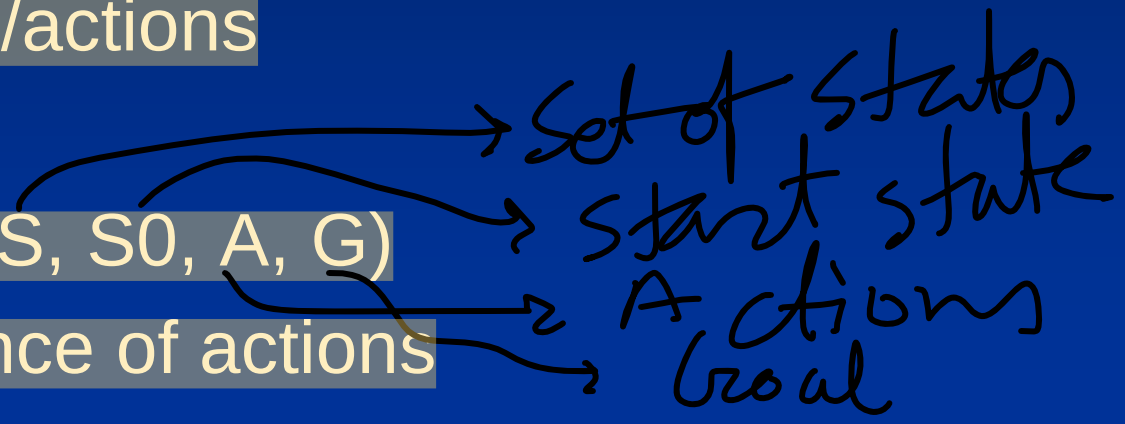
- The algorithm
- The space and time complexities
- When to select a particular strategy

Instructional Objectives

- At the end of the lecture the student should be able to do the followings:
  - Analyze a given problem and identify the most suitable search strategy for the problem
  - Given a problem and apply one of these strategies to find a solution for the problem

Search Problem representation

- S: set of states
- Initial state: s0 $\in$ S
- A: S$\rightarrow$S operators/actions
- G: Goal
- Search problem: (S, S0, A, G)
- A plan is a sequence of actions
- P= {a1, a2……an), which leads to traversing a number of states
- ({s0, s1, …..sN+1$\in$g
- Path cost: path$\rightarrow$Positive number



What is blind or Uninformed Search Algorithm?

- This is one that uses no information other than the initial state, the search operators, and a test for a solution

Breadth first search

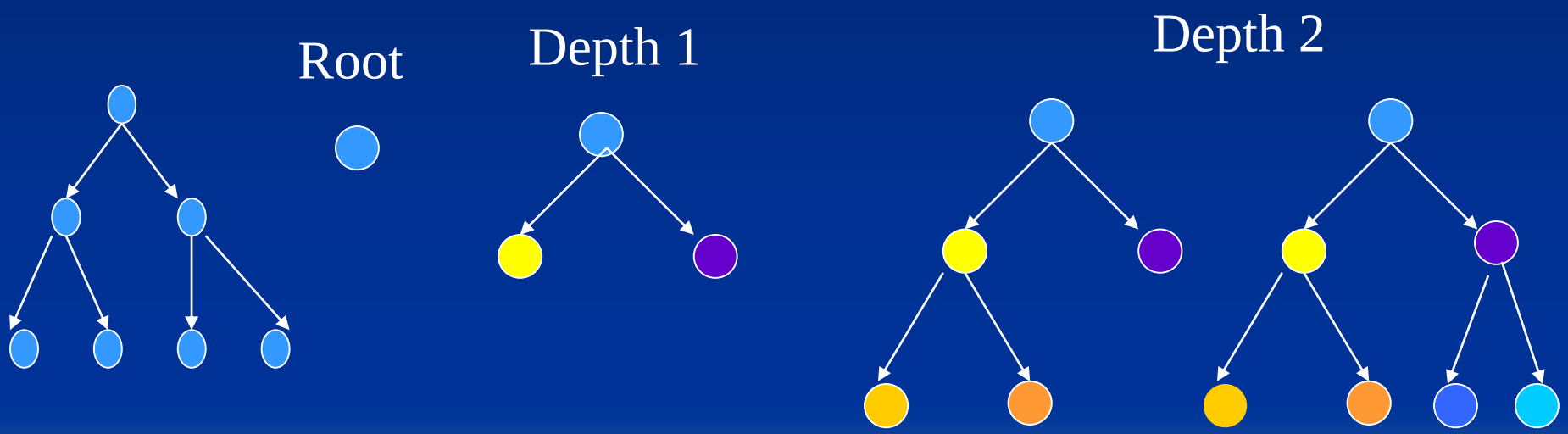
- Expand shallowest node first.
- Implementation:

– Queueing-Fn: Append new nodes to end of queue.

[This search is a very systematic strategy because it considers all the paths of length 1 first, then all those of length 2 and so on. If there is a solution, this serach is guaranteed to find it, and if there are several solutions, the search will always find the shallowest goal state first.]

Breadth-first search

1. All the nodes are expanded at a given depth before expanding nodes from the next level
2. First-in-first-out (FIFO) queue.



The BFS Algorithm

1. Place the starting node s on the queue
2. If the queue is empty, return failure and stop
3. If the first element on the queue is a goal node g, return success and stop. Otherwise
4. Remove and expand the first element from the queue and place all the children at the end of the queue in any order
5. Return to Step 2.

BFS

Let fringe be a list containing the initial state

Loop

if fringe is empty return failure
Node $\rightarrow$ remove first (fringe)
if Node is a goal
then return the path from initial state to Node
else generate all successors of node
merge the newly generated nodes into fringe
[strategy: at the back of the fringe]

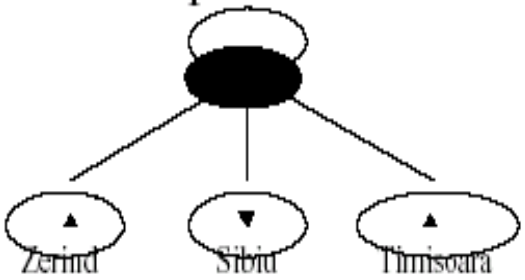
End

Note: Shallowest node expand first

Example Romania

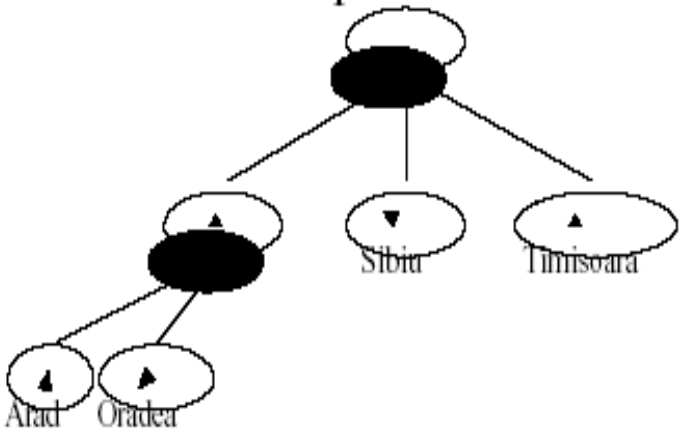
Queue: Arad

Example Romania



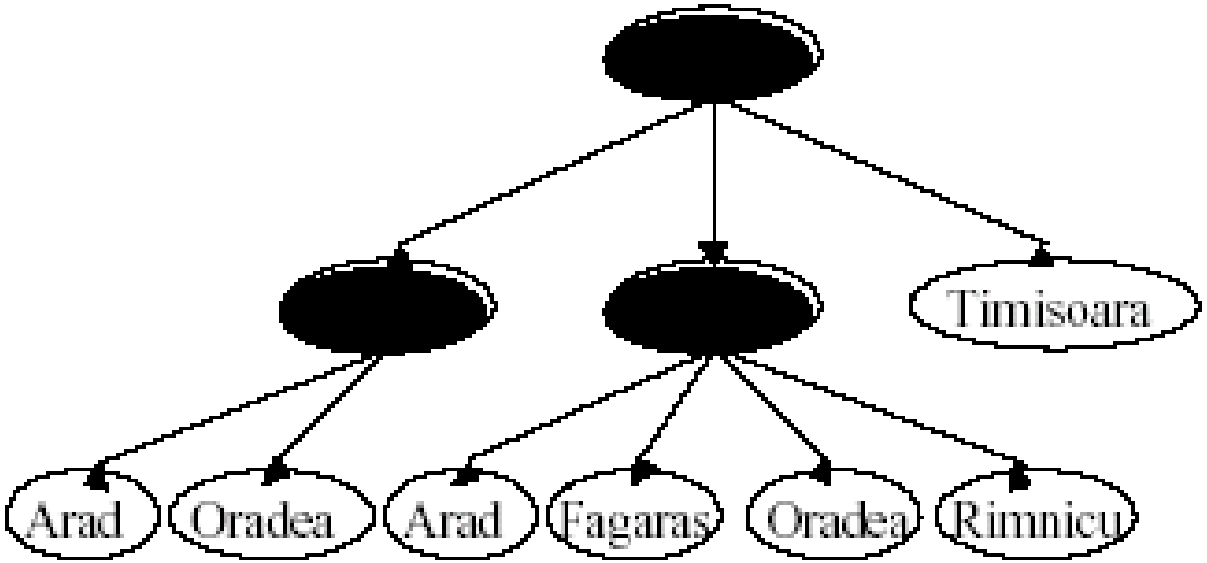
Queue: Zerind, Sibiu, Timisoara

Example Romania



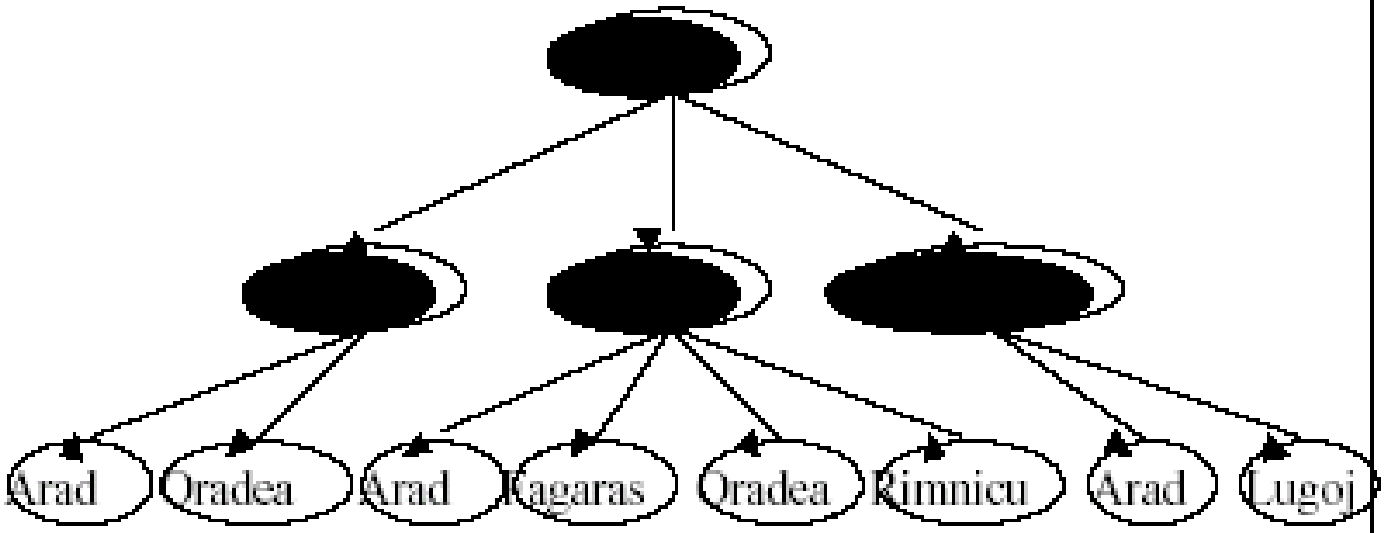
Queue: Sibiu, Timisoara, Arad, Oradea

Example Romania



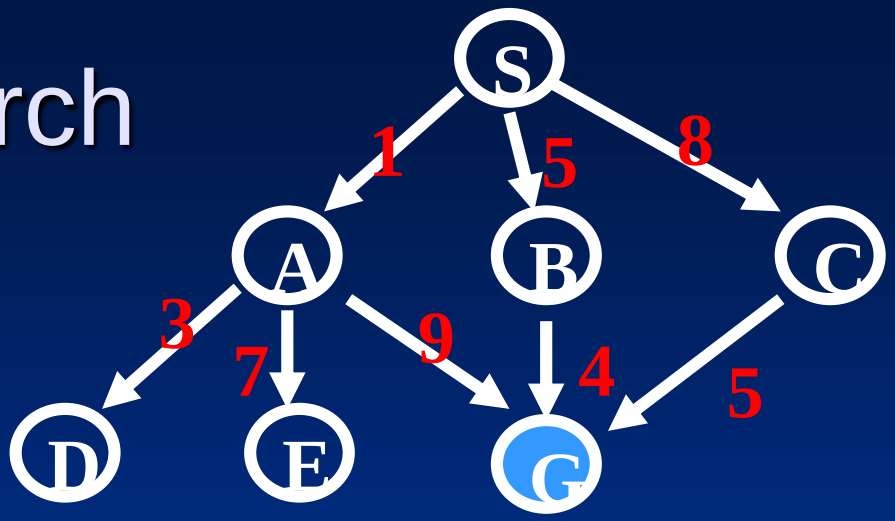
Queue: Timisoara, Arad, Oradea, Arad, Fagaras, Oradea, Rimnicu

Example Romania



Queue: Arad, Oradea, Arad, Fagaras, Oradea, Rimnicu, Arad, Lugoj

Breadth-First Search

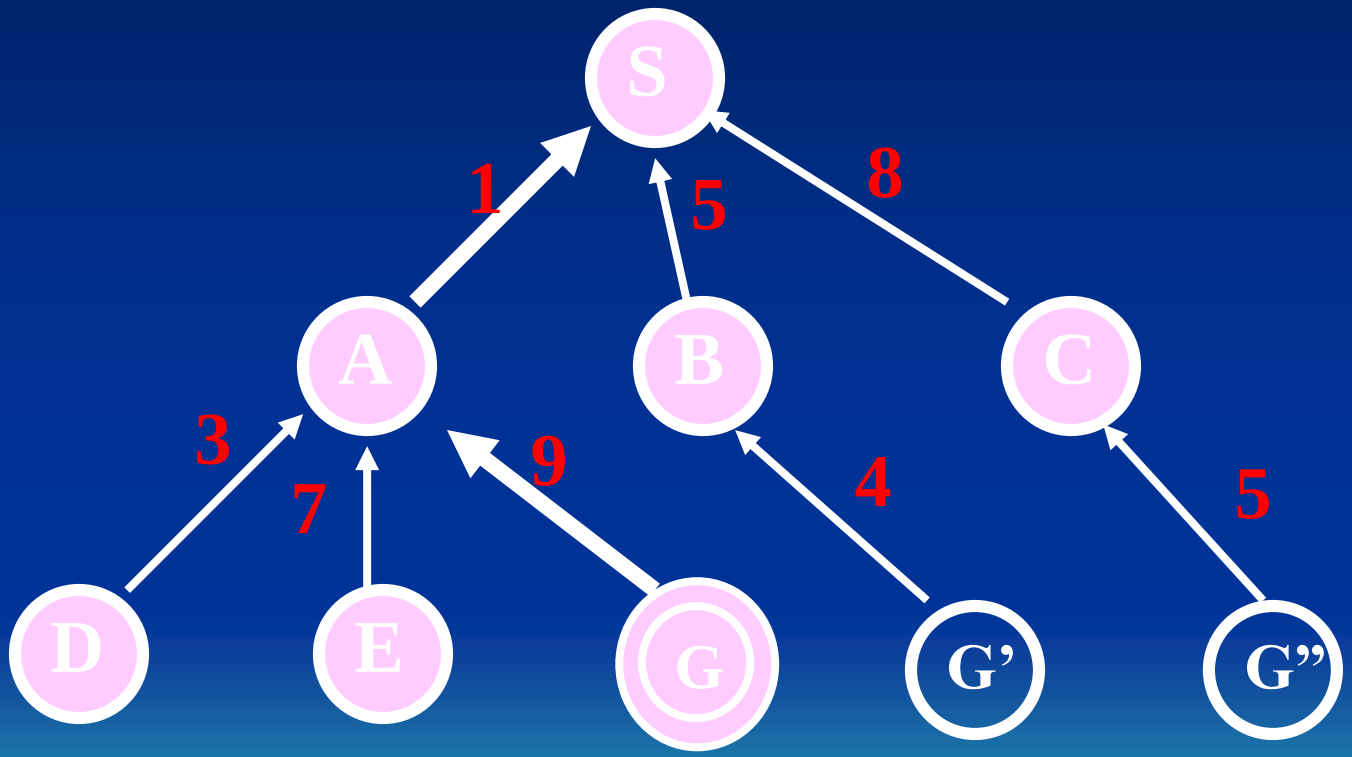


exp. node	OPEN list	CLOSED list
	{ S }	{}
S	{ A B C }	{S}
A	{ B C D E G }	{S A}
B	{ C D E G G' }	{S A B}
C	{ D E G G' G" }	{S A B C}
D	{ E G G' G" }	{S A B C D}
E	{ G G' G" }	{S A B C D E}
G	{ G' G" }	{S A B C D E}

Solution path found is S A G <-- this G also has cost 10

Number of nodes expanded (including goal node) = 7

CLOSED List: the search tree connected by backpointers



Properties of breadth first search

b = branching factor

- Completeness: Yes (if b is finite)
- Time complexity: $1+b+b^2+b^3+\ +b^d = O(b^d)$
- Space complexity: $O(b^d)$
- Optimality: Yes, if uniform cost - not optimal in general

Space is the big problem - can easily generate 1MB/sec, which results in 86GB per day - not an unusually long time.

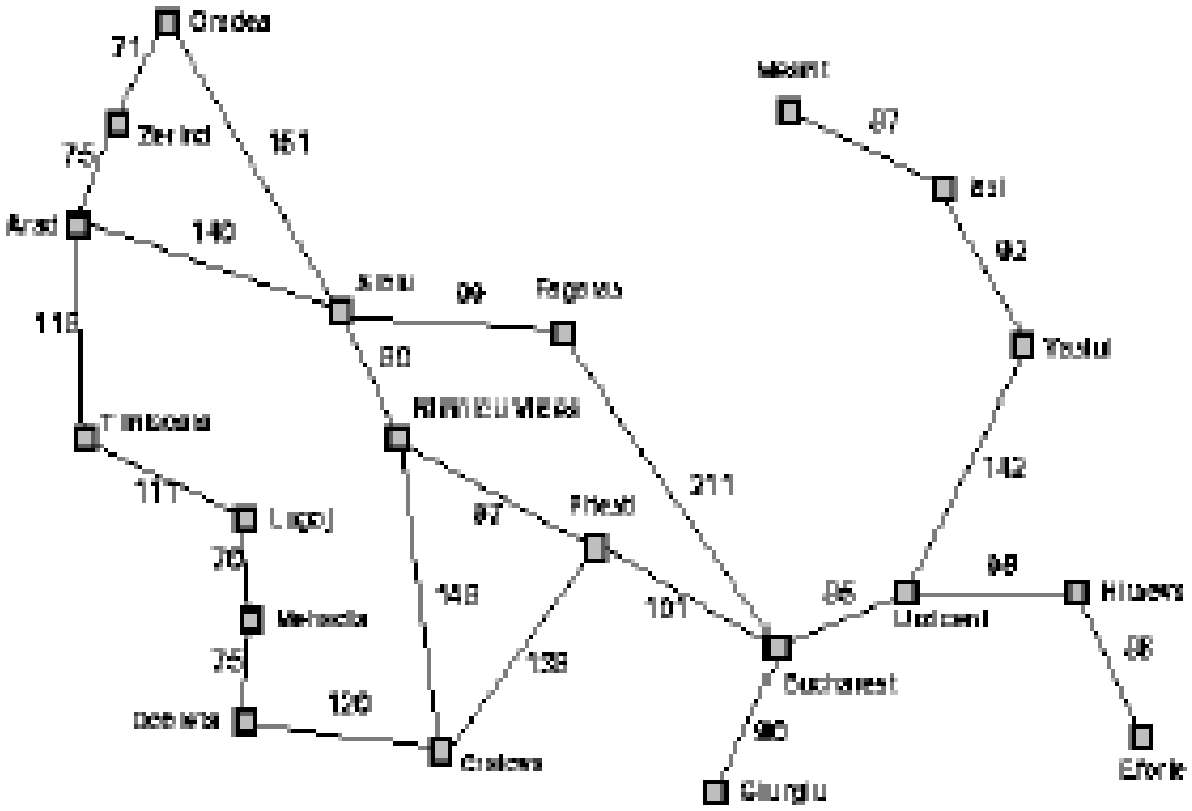
d = depth of least cost solution

Uniform cost search

- Expand least cost node first.
- Implementation:

– Queueing-Fn: Order nodes by increasing path cost.

Example Romania



Straight-line distance to Bucharest

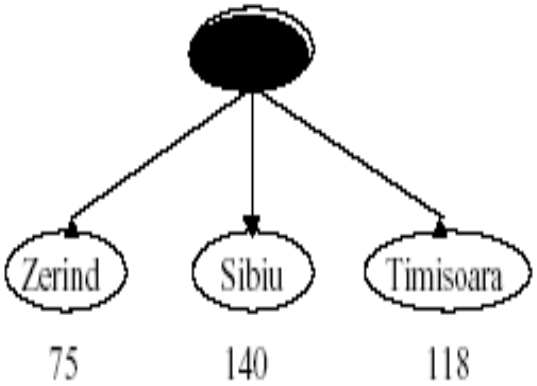
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Example Romania

0

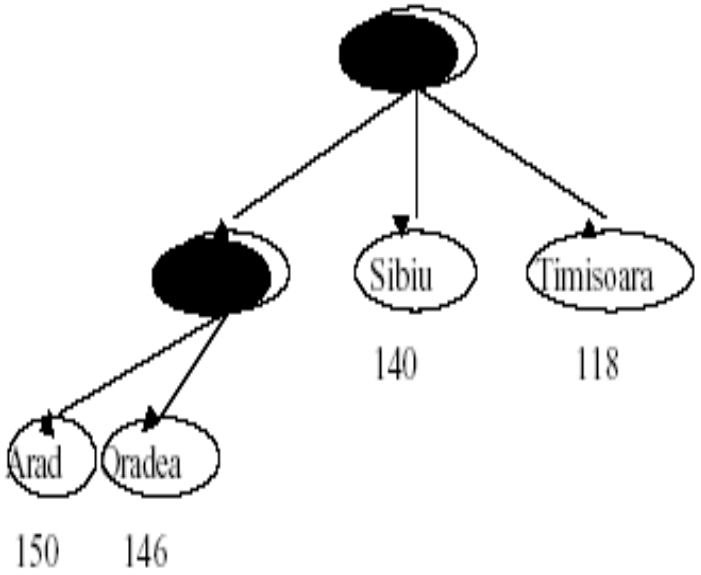
Queue: Arad

Example Romania



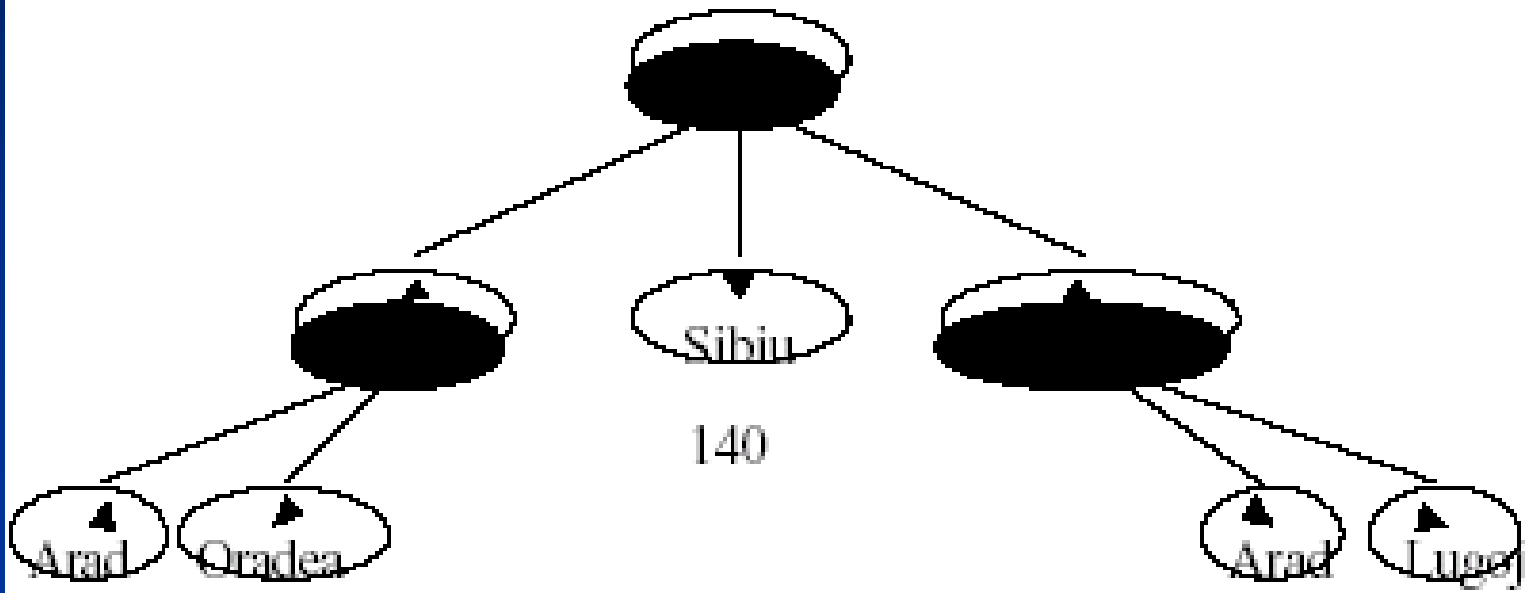
Queue: Zerind(75), Timisoara(118), Sibiu(140),

Example Romania



Queue: Timisoara(118), Sibiu(140), Oradea(146), Arad(150)

Example Romania

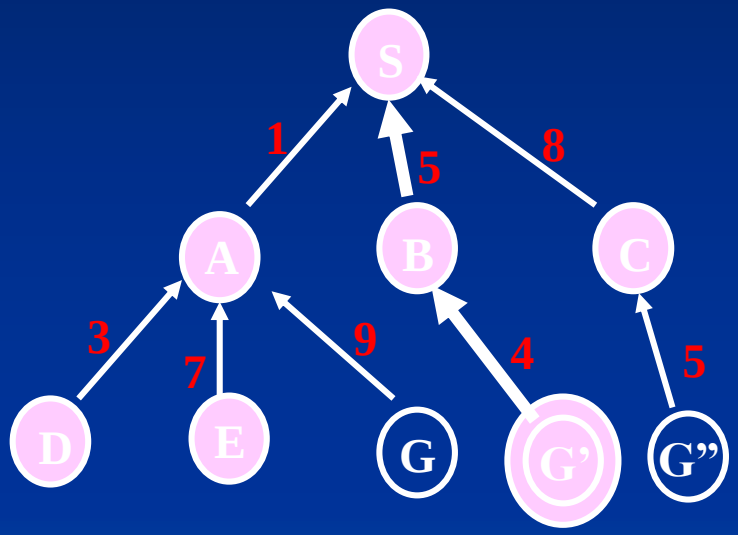


Queue: Sibiu(140), Oradea(146), Arad(150), Lugoj(229), Arad(236)

Uniform-Cost Search

GENERAL-SEARCH(problem, ENQUEUE-BY-PATH-COST)

exp. node	**nodes list**	**CLOSED list**
	{S(0)}	
S	{A(1) B(5) C(8)}	
A	{D(4) B(5) C(8) E(8) G(10)}	
D	{B(5) C(8) E(8) G(10)}	
B	{C(8) E(8) G'(9) G(10)}	
C	{E(8) G'(9) G(10) G"(13)}	
E	{G'(9) G(10) G"(13) }	
G'	{G(10) G"(13) }	



Solution path found is S B G <-- this G has cost 9, not 10

Number of nodes expanded (including goal node) = 7

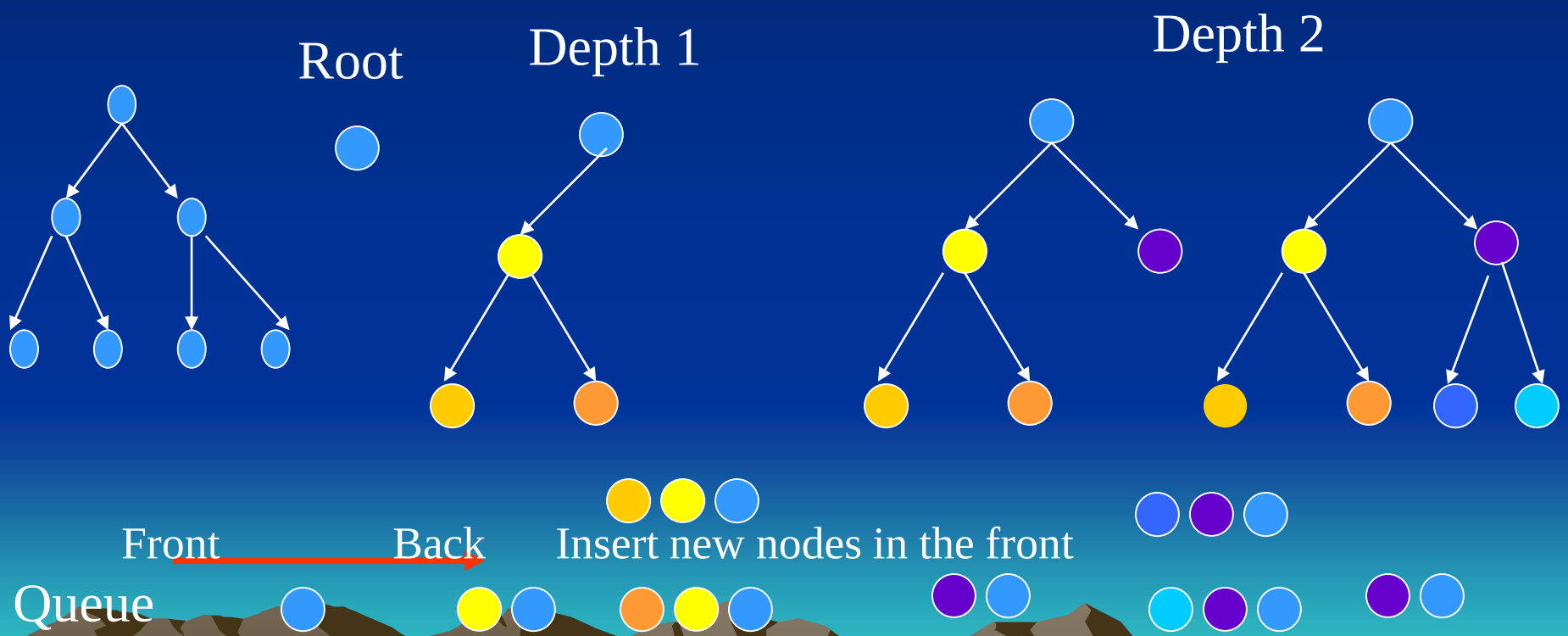
Depth first search

- Expand deepest node first.

- Implementation:
  - Queueing-Fn: Put new nodes in front of queue.

Depth-first search

1. Always expands the leaf node with greatest depth.
2. Last-in-first-out (LIFO) queue.



DFS

Let fringe be a list containing the initial state

Loop

if fringe is empty return failure
Node $\rightarrow$ remove first (fringe)
if Node is a goal
then return the path from initial state to Node
else expand deepest node first (DFS)
generate all successors of node (BFS)
if depth of node = limit return cutoff (DLS)
merge the newly generated nodes into fringe

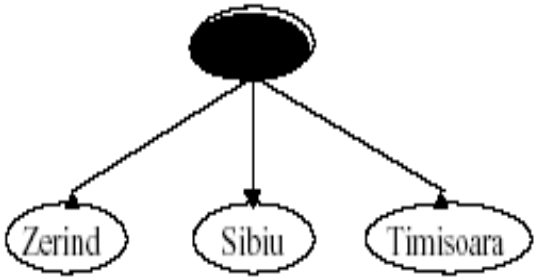
End

Note: strategy: add generated nodes to the front of the fringe

Example Romania

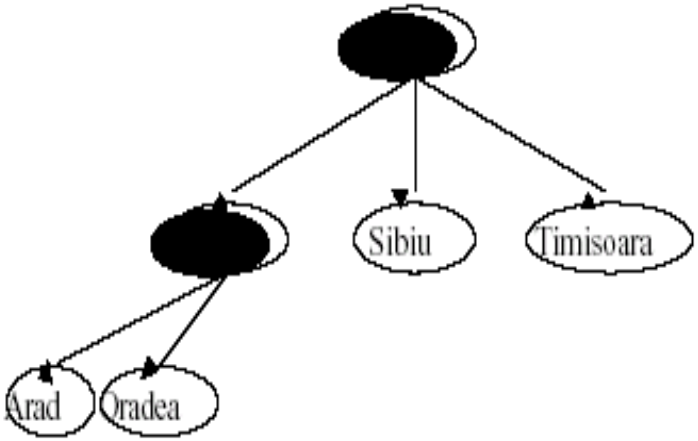
Queue: Arad

Example Romania



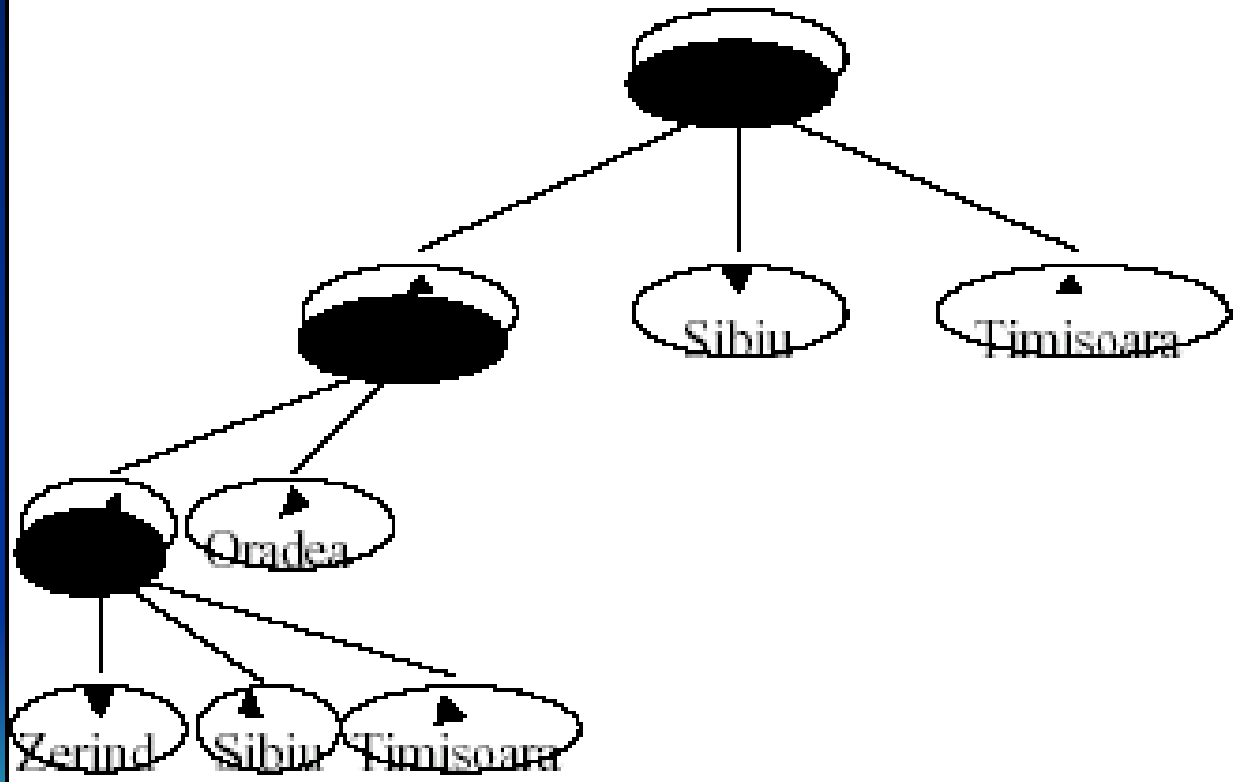
Queue: Zerind, Sibiu, Timisoara

Example Romania



Queue: Arad, Oradea, Sibiu, Timisoara

Example Romania

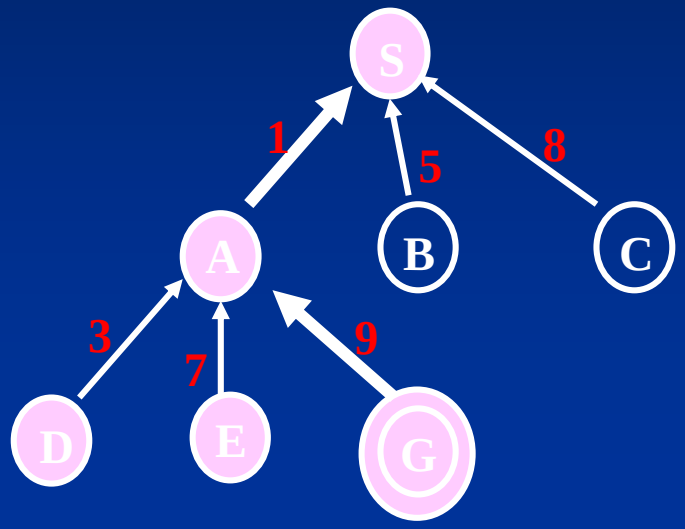


Queue: Zerind, Sibiu, Timisoara, Oradea, Sibiu, Timisoara

Depth-First Search

return GENERAL-SEARCH(problem, ENQUEUE-AT-FRONT)

exp. node	**OPEN list**	**CLOSED list**
	{ S }	
S	{ A B C }	
A	{ D E G B C}	
D	{ E G B C }	
E	{ G B C }	
G	{ B C }	



Solution path found is S A G <-- this G has cost 10

Number of nodes expanded (including goal node) = 5

m = max depth to which DFS will be searching

Properties of depth first search

- Completeness: No, in infinite depth spaces, e.g. in loops etc.
- Time complexity: $O(b^m)$, very bad where m is much larger than d.
- Space complexity: $O(bm)$, i.e. only linear
- Optimality: No.

When depth-first search is preferred

Depth limited search

- Expand deepest node first until depth limit reached.

- Implementation:
  - Queueing-Fn: Append new nodes to end of queue.

Let fringe be a list containing the initial state

Loop

if fringe is empty return failure
Node $\rightarrow$ remove first (fringe)
if Node is a goal
then return the path from initial state to Node
else generate all successors of node
merge the newly generated nodes into fringe

End

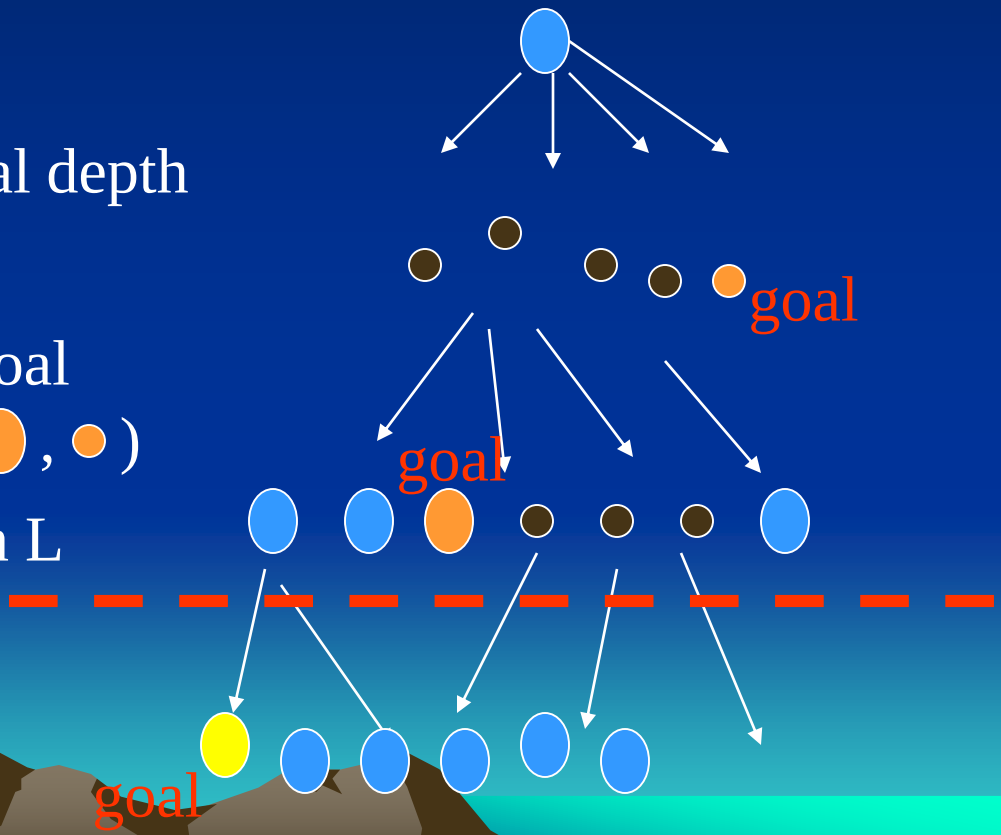
Depth-Limited search : a simple variant of depth-first search.

What to do with unbounded tree ? Just limit the allowable depth.

Given a pre-determined depth limit L,
don't search beyond depth L.

Incomplete if shallowest goal depth is d and L < d. (goal)

Non-optimal if shallowest goal depth is d and L > d. (goals ,)



Time complexity: O(b^L)

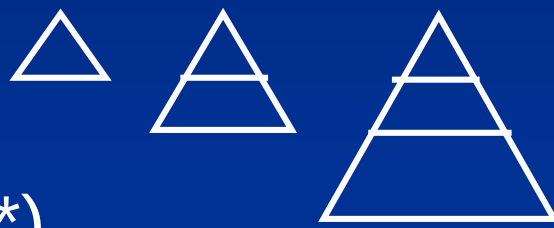
Space complexity: O(bL)

Properties of depth-limited search

- Completeness: Yes.
- Time complexity: $O(b^m)$, very bad where m is much larger than d.
- Space complexity: $O(bm)$, i.e. only linear
- Optimality: No.

Depth-First Iterative Deepening (DFID)

- BF and DF both have exponential time complexity O(b^d)
  - BF is **complete** but has exponential space complexity
  - DF has **linear space complexity** but is incomplete
- Space is often a **harder** resource constraint than time
- Can we have an algorithm that
  - Is complete
  - Has linear space complexity, and
  - Has time complexity of O(b^d)



- DFID by Korf in 1985 (17 years after A*)

  First do DFS to depth 0 (i.e., treat start node as having no successors), then, if no solution found, do DFS to depth 1, etc.

  until solution found do
  DFS with depth bound c
  c = c+1

Depth-First Iterative Deepening (DFID)

- **Complete** (iteratively generate all nodes up to depth d)
- **Optimal/Admissible** if all operators have the same cost. Otherwise, not optimal but does guarantee finding solution of shortest length (like BF).
- **Linear space complexity:** O(bd), (like DF)
- **Time complexity** is a little worse than BFS or DFS because nodes near the top of the search tree are generated multiple times, but because almost all of the nodes are near the bottom of a tree, the worst case time complexity is still exponential, O(b^d)

Depth-First Iterative Deepening

- If branching factor is b and solution is at depth d, then nodes at depth d are generated once, nodes at depth d-1 are generated twice, etc., and node at depth 1 is generated d times.

  Hence

  total(d) = b^d + 2b^(d-1) + ... + db

  <= b^d / (1 - 1/b)^2 = O(b^d).

  - If b=4, then worst case is 1.78 * 4^d, i.e., 78% more nodes searched than exist at depth d (in the worst case).

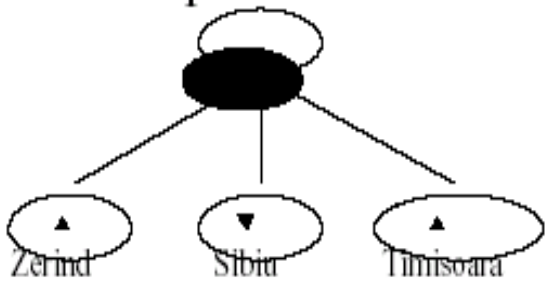
Iterative Deepening Search

```
Function Iterative-Deepening-Search(problem) return solution/failure
  for depth ← 0 to ∞ do
    result ← Depth-Limited-Search(problem, depth)
    if (result ≠ cutoff) then return result
  end.
```

Example Romania

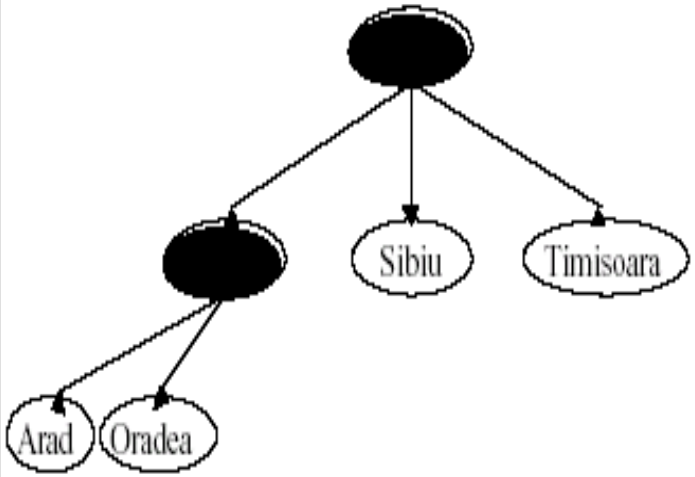
Queue: Arad

Example Romania



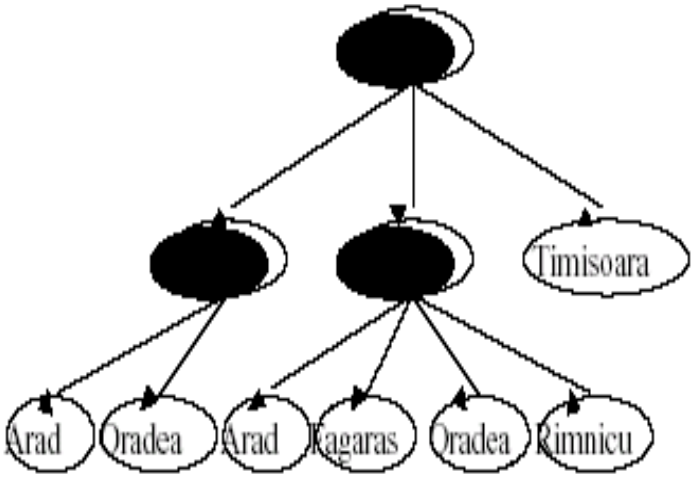
Queue: Zerind, Sibiu, Timisoara

Example Romania



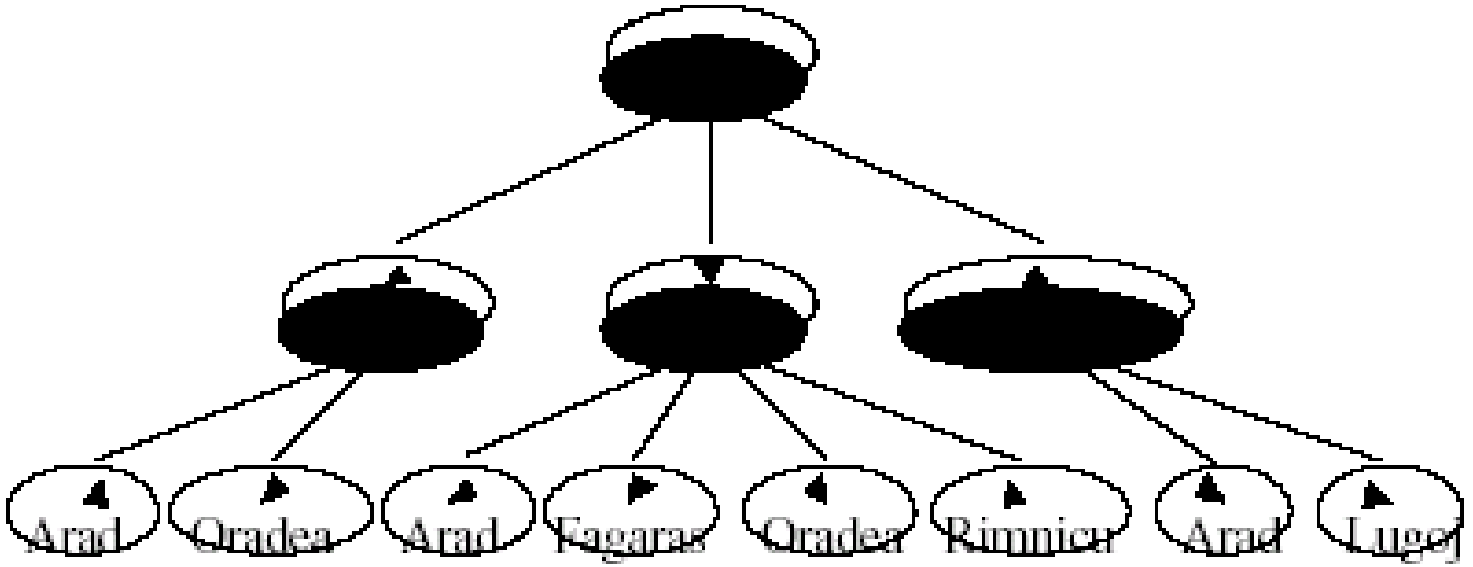
Queue: Sibiu, Timisoara, Arad, Oradea

Example Romania



Queue: Timisoara, Arad, Oradea, Arad, Fagaras, Oradea, Rimnicu

Example Romania



Queue: Arad, Oradea, Arad, Fagaras, Oradea, Rimnicu, Arad, Lugoj

Properties of Iterative Deepening Search

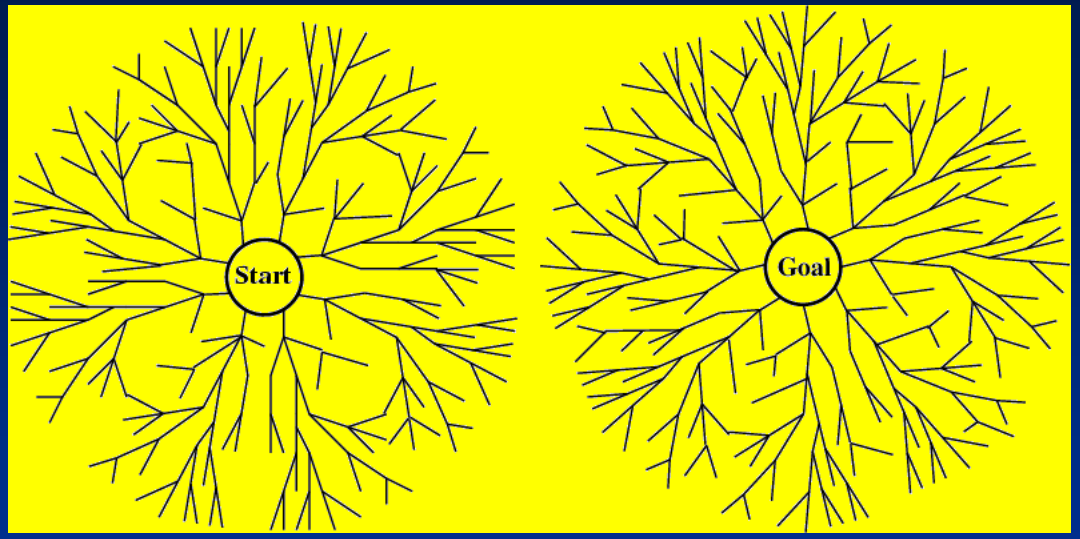
- Completeness: Yes.
- Time complexity: $(d+1)b^0+db^1+(d-1)b^2 +\ldots+ b^d=O(b^d)$
- Space complexity: $O(bd)$, i.e. only linear
- Optimality: Yes, if step cost=1.

When to use what

- **Depth-First Search:**
  - Many solutions exist
  - Know (or have a good estimate of) the depth of solution
- **Breadth-First Search**:
  - Some solutions are known to be shallow
- **Uniform-Cost Search**:
  - Actions have varying costs
  - Least cost solution is the required

  This is the only uninformed search that worries about costs.
- **Iterative-Deepening Search**:
  - Space is limited and the shortest solution path is required

Bi-directional search



- Alternate searching from the start state toward the goal and from the goal state toward the start.
- Stop when the frontiers intersect.
- Works well only when there are unique start and goal states and when actions are reversible
- Can lead to finding a solution more quickly (but watch out for pathological situations.

Comparing Search Strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Time	b^d	b^d	b^m	b^l	b^d	$b^{d/2}$
Space	b^d	b^d	bm	bl	bd	$b^{d/2}$
Optimal?	Yes	Yes	No	No	Yes	Yes
Complete?	Yes	Yes	No	Yes, if $l \geq d$	Yes	Yes

Summary

- Problem formulation usually requires abstracting away real-world details to define a state space that can be feasibly explored

- Iterative deepening uses linear space and not much more time than other uninformed search strategies.

Assignment

- Exercise 3.4,3.6,3.9,3.10,3.11,3.12,3.13,3.15,3.17

State space - Graph

- In search tree we may get to same node many times
- In tree search is exponentially larger than the graph [show how]
- If the search space is not a tree, but a graph, the search tree may contain different nodes corresponding to the same state.

Avoiding Repeated states

- In increasing order of effectiveness in reducing size of state space and with increasing computational cost:
  - Do not return to the state you just come from (8-puzzle)
  - Do not create paths with cycles in them
  - Do not generate any state that was ever created before [CLOSED LIST]
- Net effect depends on frequency of “loops” in state space

GRAPH SEARCH ALGORITHM

Let fringe be a list containing the initial state
Let closed be initially empty

Loop

if fringe is empty return failure
Node → remove first (fringe)
if Node is a goal
then return the path from initial state to Node
else put node in CLOSED
generate all successors of node S
for all nodes m in S
if m not in CLOSED
merge m into fringe
End Loop

UCS

- Enqueue nodes by path cost
- Let g(n) = cost of the path from the start node to the current node n. Sort nodes by increasing value of g
- Expand lowest cost node of the fringe
  - Complete
  - Optimal/admissible
  - Exponential time and space complexity
  - [priority queue

Example